

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 2 — Progettare il prompt

Ragionare: il chain-of-thought sui modelli di oggi

«Let’s think step by step»

Il *chain-of-thought*, nel 2022: una frase in più nel prompt

- ***Chain-of-thought***, catena di pensiero: far **ragionare il modello per passi**, scritti, prima della risposta — nel 2022 uno studio (Wei e altri) lo otteneva con alcuni esempi risolti passo per passo, nel prompt
- Un altro studio del 2022 (Kojima e altri): **nessun esempio**, una frase sola prima della risposta, «Let’s think step by step» — su un modello di allora, text-davinci-002, l’accuratezza sui problemi aritmetici di MultiArith passa dal **17,7 al 78,7 %**
- Ma solo sui modelli **grandi**: secondo Wei e altri i guadagni arrivano verso i 100 miliardi di parametri; sotto, catene di passi fluenti ma illogiche

Allora il modello ragionava scrivendo: **lo spazio per ragionare era la risposta**.

Il problema degli sportelli

M2-D05, dal vivo: «rispondi direttamente», sui tre modelli, a effort low e high

- **Indovinate, a voce**: chiedo a Haiku di rispondere **direttamente**, senza ragionare — userà **token di ragionamento**?

`sportelli.md, poi direttamente.txt`

```
In un ufficio ci sono 3 sportelli. Ogni sportello serve una
persona ogni 4 minuti. Alle 9:00 ci sono 27 persone in fila. A
che ora esce l'ultima persona?
Rispondi direttamente, senza ragionare: scrivi solo l'orario.
```

1. Poi la stessa richiesta con `MAX_THINKING_TOKENS=0` e a effort **high**; e Sonnet e Opus, a low e a high: **sette chiamate**

```
unset CLAUDE_CODE_EFFORT_LEVEL MAX_THINKING_TOKENS    # passo 2
claude -p "$(cat sportelli.md)"$'\n'"$(cat direttamente.txt)" \
  --model haiku --effort low --tools "" \
  --system-prompt "$(cat istruzione.txt)" --strict-mcp-config \
  --setting-sources project,local --output-format json \
  --no-session-persistence
# ... le altre sei chiamate e ragionamento.tsv: README, passo 3
```

- Demo **M2-D05**, dal vivo: nel JSON di ogni risposta, i **token di ragionamento** e quelli **in uscita**

Il problema degli sportelli

M2-D05, dal vivo: «rispondi direttamente», sui tre modelli, a effort low e high

Il problema degli sportelli

In un ufficio ci sono 3 sportelli. Ogni sportello serve una persona ogni 4 minuti. Alle 9:00 ci sono 27 persone in fila. A che ora esce l'ultima persona?
Rispondi direttamente, senza ragionare: scrivi solo l'orario.

la chiamata	prova 1	prova 2	prova 3
R1 Haiku, low	103 / 108 · 9:36	160 / 165 · 9:36	115 / 120 · 9:36
R2 Haiku, low, MAX_THINKING_TOKENS=0	126 / 131 · 9:36	113 / 118 · 9:36	124 / 129 · 9:36
R3 Haiku, high	163 / 168 · 9:36	116 / 121 · 9:36	120 / 125 · 9:36
R4 Sonnet, low	0 / 5 · 9:36	0 / 5 · 9:36	0 / 5 · 9:36
R5 Sonnet, high	101 / 106 · 9:36	62 / 67 · 9:36	87 / 92 · 9:36
R6 Opus, low	26 / 31 · 9:36	0 / 29 · 9:36	0 / 26 · 9:36
R7 Opus, high	33 / 40 · 9:36	33 / 38 · 9:36	34 / 39 · 9:36

In ogni casella: token di ragionamento / token in uscita · l'orario della risposta, un dato

I token di ragionamento dentro quelli in uscita: 21 chiamate su 21 (la garanzia)

Haiku, chiesto di rispondere direttamente, ha ragionato: 3 volte su 3

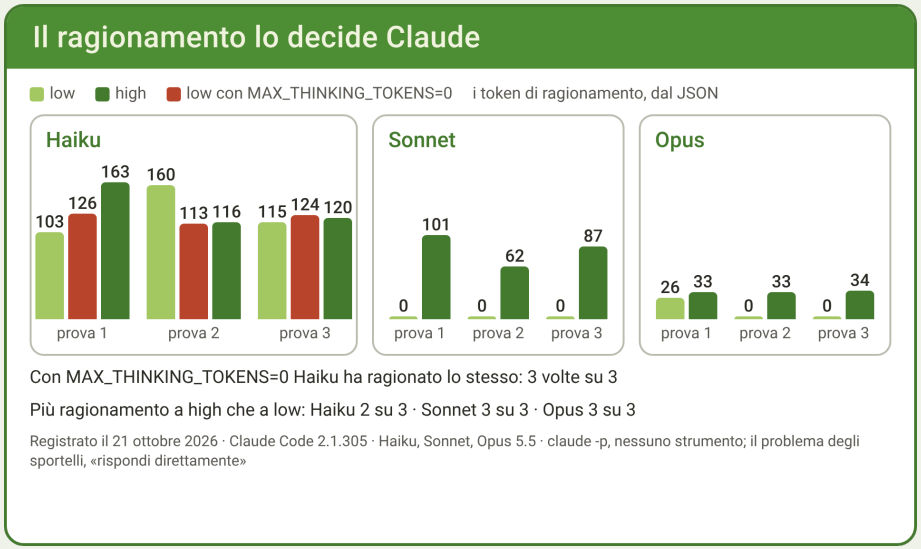
Registrato il 21 ottobre 2026 · Claude Code 2.1.305 · Haiku, Sonnet, Opus 5.5 · claude -p, nessuno strumento

Le sette chiamate, tre volte ciascuna, nessuno strumento: i token di ragionamento, quelli in uscita e l'orario della risposta, un dato
Sotto, due esiti del controllo
© Simone Scannapieco — figura dalla registrazione del 21 ottobre 2026

- Anthropic, per Haiku 5.5: «Nei test di Anthropic, dire al modello nel prompt di rispondere direttamente non gli ha impedito di ragionare» — nella registrazione Haiku ha ragionato **3 volte su 3**, da 103 a 160 token
- Il ragionamento si paga come **token in uscita**: `thinking_tokens` «è sempre minore o uguale a `output_tokens`», scrive Anthropic — nella registrazione, **21 chiamate su 21**

Il ragionamento lo decide Claude

Sui modelli 5.5 il *thinking* non si spegne: la leva è l'effort



I token di ragionamento, tre prove per modello: a effort low, a high, e Haiku a low con MAX_THINKING_TOKENS=0

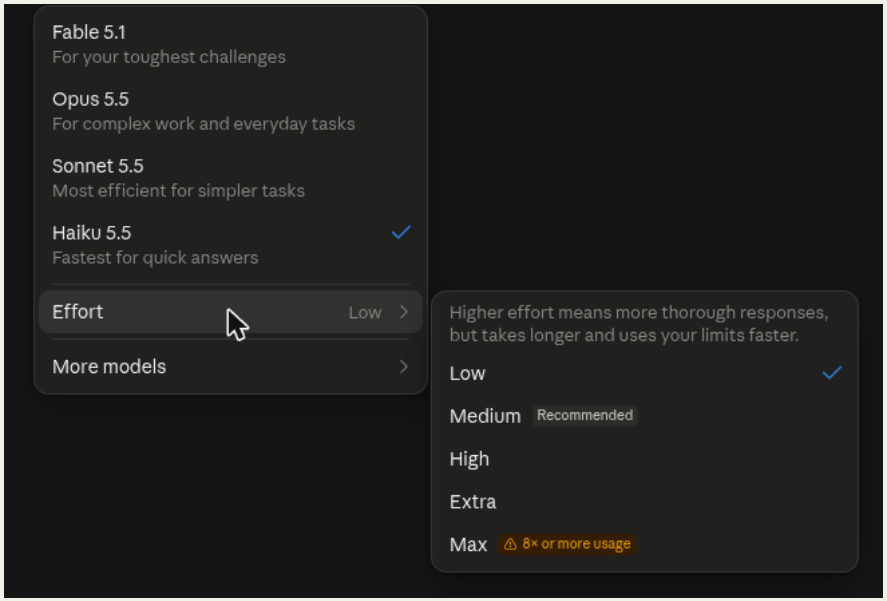
Sotto, due esiti del controllo

© Simone Scannapieco — figura dalla registrazione del 21 ottobre 2026

- Anthropic: «Il ragionamento è **facoltativo** per il modello»: a ogni richiesta Claude valuta la complessità e decide se ragionare — nessun livello di effort lo garantisce
- Sui modelli 5.5 il *thinking* **non si spegne**, né in Claude Code né in claude.ai: con `MAX_THINKING_TOKENS=0` Haiku ha ragionato lo stesso **3 volte su 3**
- «Un effort più elevato induce **più ragionamento**» (Anthropic) — da low a high più token di ragionamento: Haiku **2 su 3**, Sonnet **3 su 3**, Opus **3 su 3**

Il ragionamento lo decide Claude

Sui modelli 5.5 il *thinking* non si spegne: la leva è l'effort



Il menu del modello in claude.ai, accanto al pulsante di invio: i modelli e, nel sottomenu, i livelli dell'effort

Schermata di claude.ai del 20 ottobre 2026, dall'account del docente (piano Max)

- La leva è l'**effort**: `--effort` o `/effort` in Claude Code, il menu del modello in claude.ai — più effort, più ragionamento, più token: i limiti d'uso arrivano prima
- In Claude Code, `ultrathink` nel prompt chiede più ragionamento **per quel turno**, senza cambiare l'effort; «think hard» invece è testo come un altro

Che cosa consiglia Anthropic

Sul ragionamento: quattro consigli, dalla documentazione e dalla ricerca di Anthropic

Istruzioni generali, non passi prescrittivi — nella documentazione, in italiano:

> *Preferisci istruzioni generali a passaggi prescrittivi. Un prompt come "ragiona a fondo" spesso produce un ragionamento migliore rispetto a un piano passo passo scritto a mano.*
>
> **Documentazione di Anthropic, versione italiana**

- Il ragionamento scritto si chiede **per leggerlo**: il passo passo chiesto a mano è un ripiego per quando il *thinking* è spento — e su Opus e Sonnet 5.5 la richiesta di scrivere il proprio ragionamento **può essere rifiutata**
- Una **breve spiegazione** va sempre bene: le evidenze dietro un risultato. E il ragionamento si legge anche senza chiederlo: in claude.ai la sezione «Thinking» ne mostra un riassunto, in Claude Code `Ctrl+O`
- Il ragionamento mostrato **non è una prova**: Anthropic avverte che non è certo che rappresenti ciò che accade davvero nel modello — leggetelo come un indizio, e verificate il risultato

Le strategie, in una slide

Da nessuna istruzione allo schema — sei modi di chiedere

Implicit	solo i dati, senza istruzione: il modello indovina che cosa volete — ogni ingrediente che manca è una decisione lasciata a lui
Zero-shot	istruzione e dati, nessun esempio — il default
Few-shot	dopo l'istruzione, alcuni esempi: insegnano la forma — le etichette e le regole si scrivono
Role-based	ciò che non cambia da una richiesta all'altra nel *system prompt*: ruolo, regole, formato; il caso e i dati nel messaggio
***Thinking* ed effort**	il ragionamento lo decide Claude: si regola con l'effort, e quello scritto si chiede per leggerlo
Output strutturato	la forma della risposta imposta da uno schema — quando il risultato entra in un programma

Quale strategia per quale compito

La tabella dei criteri — anche nella scheda `riferimenti/pattern-di-prompt.md`

Compito	Strategia	Perché
Riassumere, tradurre	*Zero-shot*, con formato e vincoli	compito comune: basta dire come lo volete
Classificare, estrarre	Gli esempi e le etichette ammesse, o uno schema	gli esempi insegnano la forma; le etichette si scrivono
Calcolare, decidere	Lasciarlo ragionare, con l'effort più alto; leggere il ragionamento quando conta	il ragionamento lo decide Claude: si regola, e si legge quando serve
Rispondere ai clienti	Ruolo, contesto, procedure, esempi di stile	il tono si mostra; che cosa si può promettere lo dicono le procedure
Alimentare un programma	Uno schema JSON: in Claude Code `--json-schema`	lo schema garantisce la forma, non il contenuto

Sui modelli di oggi il ragionamento lo decide Claude: la leva è l'effort, e il ragionamento si chiede per leggerlo.